סיכום שיעור 6: ShopList בתוך Kubernetes

Nov 13, 2026 · @Rivka

התמונה הגדולה

בשיעור 6 העברנו את כל ShopList מ-Docker Compose ל-Kubernetes. כתבנו 12 קבצי YAML בתיקייה kubernetes, ובפקודה אחת האפליקציה עלתה ועובדת בכתובת localhost:30080.

המשפט של השיעור: "ה-pod זמני. כל מה שחשוב גר מחוץ לו."

כל pod יכול להימחק בכל רגע, ו-Kubernetes מקים במקומו חדש. לכן כל מה שצריך לשרוד יצא מה-pod למשאב משלו: ההגדרות, הסיסמה, הנתונים והכתובת.

מפת ShopList בתוך ה-cluster

ארבעה סרוויסים, כמו ב-Docker Compose, וכולם באותו namespace, שנקרא shoplist:

מה מחובר אליו	פורט	מה הוא עושה	סרוויס
דלת מבחוץ: 30080	80	מגיש את האתר, ומעביר בקשות הלאה	ה-frontend
מדבר עם postgres	5000	ה-API של המוצרים, ב-Python	ה-backend
מדבר עם ה-backend	3000	סטטיסטיקות, ב-Node	ה-stats
הארונית postgres-pvc	5432	שומר את המוצרים	מסד הנתונים postgres

הדפדפן מדבר רק עם ה-frontend, דרך הדלת 30080. כל שאר הסרוויסים מדברים ביניהם בתוך ה-cluster, לפי השמות של ה-Services.

האנלוגיה: הקונדיטוריה משיעור 5

- ה-cluster הוא רשת קונדיטוריות.
- כל node הוא מטבח.
- כל pod הוא מגש, וה-container הוא העוגה שעליו.
- ה-Deployment הוא הזמנה קבועה: "תמיד שלושה מגשים על הדלפק".
- ה-scheduler הוא הסדרנית, שמחליטה לאיזה מטבח כל מגש הולך.

המשאבים: מה כל אחד פותר

כל משאב קיים כי בלעדיו משהו נשבר. השאלה "מה היה קורה בלעדיו?" היא הדרך הכי טובה לזכור למה הוא שם.

האנלוגיה	מה קורה בלעדיו	הבעיה שהוא פותר	המשאב
המחלקה של ShopList	הכל מעורבב, ומחיקה אחת פוגעת באחרים	שתים-עשרה משאבים בתוך cluster שיש בו עוד דברים	ה-Namespace
ההזמנה הקבועה	שנמחק נעלם לתמיד pod	ה-pod לבד מת ולא קם	ה-Deployment
מספר טלפון קבוע	ה-backend לא מוצא את postgres אחרי שה-pod מתחלף	לכל pod כתובת חדשה בכל לידה	ה-Service
הדף על המקרר	בונים image חדש לכל שינוי הגדרה	קובץ הגדרות צריך להגיע ל-container	ה-ConfigMap
מעטפה בכתב ראי	הסיסמה גלויה לכל מי שקוראת את ה-Deployment	סיסמה לא צריכה להיות כתובה ב-Deployment	ה-Secret
טופס בקשה לארונית	כל המוצרים נעלמים כש-postgres מתחלף	הדיסק של pod נמחק איתו	ה-PVC

מדבקות: labels ו-selectors

ל-pods אין שם קבוע. הם נוצרים ונמחקים כל הזמן. לכן ה-Deployment וה-Service מוצאים אותם לפי מדבקה, למשל app: backend.

- ה-Deployment מדביק את המדבקה על כל pod שהוא יוצר, ומזהה את ה-pods שלו לפיה.
- ה-Service שולח שיחות לכל pod שיש עליו את המדבקה שב-selector שלו.

טעות נפוצה: מדבקה שכתובה קצת אחרת ב-Service. ה-Service נוצר בלי שגיאה, אבל אין מאחוריו אף pod. בודקות עם `kubectl get endpoints -n shoplist`: אם הרשימה ריקה, אין מאחורי ה-Service אף pod.

שני סוגי Services של ShopList

ב-ShopList	מי יכולה להתקשר	הסוג
postgres, backend, stats	רק מתוך ה-cluster: שלוחה פנימית	הסוג ClusterIP, שהוא ברירת המחדל
ה-frontend, בדלת 30080	גם מבחוץ, דרך דלת קבועה במחשב	הסוג NodePort

בענן משתמשים בסוג שלישי, LoadBalancer, או ב-Ingress: כניסה אחת לכל ה-cluster, ששולחת כל בקשה ל-Service הנכון לפי שם האתר. הרעיון: דלת ציבורית אחת, וכל השאר פנימי.

הסיסמה: מעטפה, לא כספת

הערכים ב-Secret כתובים ב-base64. זה קידוד, לא הצפנה: כל מי שקוראת את ה-Secret מפענחת אותו בפקודה אחת. ההגנה האמיתית היא הרשאות: מי מורשית לקרוא Secrets.

ובאותו היגיון: סיסמה לא שמים ב-ConfigMap. ה-ConfigMap הוא דף על המקרר: כל מי שנכנסת למטבח קוראת אותו.

הארונית: הבקשה ומה שמקבלים

ב-Kubernetes	במלון	
ה-PVC	טופס: "ארונית בגודל בינוני"	הבקשה
ה-PV	הארונית עצמה	מה שמקבלים
ה-StorageClass, שב-Docker Desktop נקרא hostpath	המחסן	מי מייצר ארוניות

ה-pod של postgres מחבר את הארונית לתיקייה של הנתונים. ה-pod נמחק? הארונית נשארת, ו-pod חדש מתחבר אליה. מה שמוחק את הנתונים הוא רק מחיקה של ה-PVC עצמו.

איך קוראים קובץ YAML

ארבע קומות בכל קובץ

כל קובץ של Kubernetes בנוי מאותן ארבע קומות:

```
apiVersion: apps/v1     # which API version
kind: Deployment        # what kind of resource
metadata:               # name, namespace, labels
  name: backend
  namespace: shoplist
spec:                   # the desired state
  replicas: 1
```

ארבע שאלות לכל קובץ

בכל פעם שפותחות קובץ, שואלות את אותן ארבע שאלות, לפי הסדר:

1. **מה זה?** התשובה בשורות apiVersion ו-kind. למשל: Service, מספר טלפון קבוע.
2. **איך קוראים לו?** התשובה ב-metadata.name. למשל: backend. השם הוא גם הכתובת.
3. **מה הוא רוצה?** התשובה ב-spec. למשל: להעביר שיחות לפורט 5000.
4. **את מי הוא מוצא, ומי מוצא אותו?** מחפשות selector, שמות שנגמרים ב-Ref, את השדה claimName, וכל שם שמופיע גם בקובץ אחר.

שאלה 4 היא הכי חשובה: קובץ ב-Kubernetes אף פעם לא עומד לבד. כשמשהו לא עובד, כמעט תמיד הסיבה היא שם שכתוב קצת אחרת בשני מקומות.

השמות שמחברים בין הקבצים

שמות שחייבים להיות זהים בדיוק בשני מקומות, ומה קורה כשהם לא:

מה קורה אם הוא לא זהה	משתמשים בו ב	מוגדר ב	השם
ה-pod תקוע ב-CreateContainerConfigError	ה-Deployments של postgres ושל ה-backend	הקובץ postgres-secret.yml	ה-Secret בשם postgres-secret
ה-pod תקוע ב-ContainerCreating	ה-Deployment של postgres	הקובץ postgres-configmap.yml	ה-ConfigMap בשם postgres-init
ה-pod תקוע ב-Pending	השדה claimName ב-postgres	הקובץ postgres-pvc.yml	הארונית postgres-pvc
ה-Service ריק, בלי endpoints	ה-selector של ה-Deployment ושל ה-Service	ה-template של ה-Deployment	המדבקה app: backend

מה קורה אם הוא לא זהה	משתמשים בו ב	מוגדר ב	השם
ה-backend לא מתחבר למסד הנתונים	המשתנה DB_HOST ב-backend	הקובץ postgres-service.yml	ה-Service בשם postgres
ה-frontend נופל: host not found	הקובץ nginx.conf של ה-frontend, והמשתנה BACKEND_URL ב-stats	הקבצים של ה-Services	ה-Services בשם backend ו-stats

בדיקות בריאות, שמירת נתונים ותיקון עצמי

השלט על הדלת, ובדיקת הדופק

שתי בדיקות ש-Kubernetes מריץ על כל pod, כל כמה שניות:

הבדיקה livenessProbe	הבדיקה readinessProbe	
"את בכלל חיה?"	"את מוכנה לקבל לקוחות?"	השאלה
ה-container מופעל מחדש	ה-Service מפסיק לשלוח אליו שיחות. לא מפעילים מחדש	כשנכשלת
תהליך שנתקע: לא קרס, אבל לא עונה	עלייה, עומס, רכיב שעוד לא מוכן	בשביל מה
אופה שנרדמה ליד התנור: מעירים אותה	השלט "פתוח/סגור"	בקונדיטוריה

כל עוד הבדיקה נכשלת, ה-pod מסומן 0/1. כשהיא מצליחה, 1/1, והשלט מתהפך ל"פתוח".

הבדיקה	הסרוויס
הפקודה pg_isready	postgres
בקשה לכתובת /health בפורט 5000	ה-backend
בקשה לכתובת /health בפורט 3000	ה-stats

ואין סדר עלייה, כמו depends_on ב-Compose. אם ה-backend עולה לפני postgres, הבדיקה שלו פשוט נכשלת עוד כמה פעמים, ו-Kubernetes ממשיך לנסות עד שהכל מוכן.

שלוש הוכחות שזה באמת Kubernetes

1. **המוצרים שורדים.** מוחקים את ה-pod של postgres, והמוצרים עדיין שם, בזכות הארונית.
2. **המטבח מתקן לבד.** מוחקים pod, וה-Deployment מקים במקומו חדש תוך שניות. זו ההזמנה הקבועה.
3. **משנים את ההזמנה דרך הקובץ.** משנים בקובץ ל-3 :replicas, מחילים שוב, ורואים שלושה pods. הקובץ הוא מקור האמת.

```
kubectl delete pod -n shoplist -l app=backend
kubectl get pods -n shoplist -w
# a new backend pod appears within seconds
```

מילון פקודות: Compose מול kubectl

ב-Kubernetes	ב-Compose	מה רוצות
`kubectl apply -f kubernetes/`	`docker compose up -d`	להרים הכל
`kubectl get pods -n shoplist`	`docker compose ps`	לראות מה רץ
`kubectl logs deployment/backend -n shoplist`	`docker compose logs backend`	לוגים
`kubectl exec -it deployment/backend -n shoplist -- sh`	`docker compose exec backend sh`	להיכנס פנימה
`kubectl delete deployment,service -l app=backend -n shoplist`	`docker compose down`	להוריד ולהשאיר נתונים
`kubectl delete pvc postgres-pvc -n shoplist`	`docker compose down -v`	להוריד ולמחוק נתונים

ועוד פקודות שכדאי להכיר:

```
kubectl get pods -n shoplist -w          # watch pods change live
kubectl describe pod -n shoplist -l app=backend   # Events at the bottom
kubectl get endpoints -n shoplist        # which pods are behind each Service
kubectl get svc -n shoplist              # frontend should show 80:30080/TCP
kubectl get pvc -n shoplist              # postgres-pvc should be Bound
```

זהירות: הפקודה `kubectl delete -f kubernetes/` מוחקת גם את הארונית וגם את ה-namespace, כי שניהם בתיקייה. בסוף שיעור 6 לא מוחקות כלום: ממשיכות מכאן בשיעור 7.

תקלות נפוצות

כשמשהו לא עובד, שלוש פקודות פותרות כמעט הכל: `kubectl get pods` כדי לראות את המצב, `kubectl describe pod` כדי לקרוא את ה-Events למטה, ו-`kubectl logs` כדי לראות מה האפליקציה אמרה.

מה עושים	הסיבה	מה רואים
לבדוק עם `docker images shoplist-*`. השם חייב להיות זהה, כולל local	ה-image לא נבנה, או שהשם לא תואם	המצב ErrImageNeverPull
ב-describe כתוב בדיוק איזה שם לא נמצא	שם ה-Secret או שם המפתח כתוב אחרת	המצב CreateContainerConfigError
ב-describe, לחפש למטה Readiness probe failed	בדיקת המוכנות נכשלת	ה-pod תקוע ב-0/1
לקרוא את הלוג של ה-backend	חסר משהו: Secret, מפתח, או postgres	המצב CrashLoopBackOff ב-backend
להחיל אותו. ה-frontend יתאושש לבד	ה-Service של stats או של ה-backend עוד לא קיים	ה-frontend נופל, ובלוג host not found
לבדוק עם `kubectl get storageclass`	אין StorageClass	הארונית תקועה ב-Pending
למחוק את ה-PVC ולהחיל שוב. זה מוחק נתונים!	קובץ init.sql רץ רק על ארונית ריקה	השגיאה relation products does not exist
לבדוק עם `kubectl get svc -n shoplist`	ה-Service לא NodePort, או פורט אחר	הדפדפן לא נפתח ב-30080
לבדוק endpoints, ואת הלוג של stats	ה-Service של stats חסר, או BACKEND_URL שגוי	הנורית של stats אדומה

השגיאה הכי מסוכנת היא זו שלא צועקת. מדבקה שגויה ב-Service לא נותנת שום שגיאה. רק `kubectl get endpoints` מראה שאין מאחוריו אף pod.

שאלות לחזרה ולראיון

קודם לנסות לענות בעל פה, ורק אז לקרוא את התשובה.

1. מה ההבדל בין ConfigMap ל-Secret? שניהם מעבירים הגדרות ל-pods בלי לבנות image מחדש. ה-ConfigMap מיועד להגדרות רגילות, בטקסט גלוי. ה-Secret מיועד למידע רגיש: הערכים ב-base64, ואפשר להגביל עליו הרשאות בנפרד.

2. האם ה-Secret מוצפן? לא. הקידוד base64 הוא לא הצפנה. ההגנה היא הרשאות, ובסביבות אמיתיות גם כלי חיצוני לניהול סודות.

3. מה קורה לנתונים כשה-pod נמחק? כלום. הארונית, PVC, נשארת, וה-pod החדש מתחבר אליה. הנתונים נמחקים רק כשמוחקים את ה-PVC.

4. מה עושה readinessProbe, ובמה הוא שונה מ-depends_on? הוא בודק אם ה-pod מוכן לקבל בקשות, וכל עוד לא, ה-Service לא שולח אליו. ב-Kubernetes אין סדר עלייה: כל רכיב עולה, נכשל אם צריך, ומנסה שוב.

5. איך Service יודע לאילו pods לשלוח? לפי מדבקות. ה-selector שלו מחפש pods עם אותה מדבקה, ומעדכן את רשימת הכתובות כל הזמן.

6. מה ההבדל בין readinessProbe ל-livenessProbe? הראשונה שואלת "מוכנה?", וכשהיא נכשלת מפסיקים לשלוח בקשות. השנייה שואלת "חיה?", וכשהיא נכשלת מפעילים מחדש את ה-container.

7. איך מגיעים לאפליקציה מבחוץ בענן? דרך מאזן עומס אחד עם כתובת ציבורית, ואחריו Ingress ששולח כל בקשה ל-Service הנכון. כל ה-Services מאחוריו פנימיים. בלי Ingress Controller מותקן, קובץ Ingress לא עושה כלום.